Computer Architecture Lecture Notes

Anirudh Dash (EE21BTECH11002)
Riya Ann Easow (EE21BTECH11044)

August 22, 2024

Is it necessary for a function to save all registers that it modifies and then restore them back? What if the registers that the function is using are not needed by the caller function? Such save-restores are an overhead encountered by functions and we should look for means to reduce this overhead.

1 Caller vs. Callee

- **Callee:**
  - The function or procedure that is invoked by another function.
  - It performs a specific task and can return a value to the calling function.
- **Caller:**
  - The function or procedure that initiates the call to another function (the callee).
  - For example, in a program, if `main()` invokes another function `f1()`, then `main()` is caller while `f1()` is the callee.
- During nested function calls, a callee might call another function and thus play the role of both a callee and a caller.

2 Local Data on Stack

2.1 Introduction

- During recursive function calls, sometimes the stack space allocated for a program can get exhausted, which may lead to segmentation faults. This can often be prevented by ensuring that functions have proper terminating conditions during recursion. Also, there are OS methods provided to increase the size of the stack, if needed to avoid such segmentation faults.
- **Optimization:** Can we reduce the number of registers saved to memory on a function call? If a register was not previously used, then storing and restoring it might be unnecessary.
- The compiler typically knows which registers are used by the caller and callee. This can be leveraged to resolve this issue in most cases. However, there are two issues related to relying on compiler for this purpose:

1. A function might get called from multiple locations and a different set of registers might be active in each of these locations. Thus, it is not easy for a compiler to always eliminate redundant save-restores.
2. When using pre-compiled libraries, the compiler may not know which registers are used within the library and hence it might have to consider all registers used to be saved.

2.2 Should registers be saved by the Caller or the Callee?

The decision depends on the registers in use and those modified by the procedure. Let's consider two scenarios:

1. **Caller saves all registers:**
   - **Advantage:**
     - This approach can optimize the number of times registers are saved when calling functions in consecutive steps (back-to-back).
     - For example:

```
main(){
    f1();
    f2();
}
```

     - Here, the caller can optimize by saving the registers in use once before `f1()` is called and restore them only after `f2()` is completed.
   - **Limitation:**
     - Not all registers can be safely stored by the caller.
     - For example, in the instruction `jal x1, L1`, the register `x1` is used to store the return address. Since this is updated at the last step of the call, callee must save it before using it for any other purpose.
2. **Callee saves all registers:**
   - While this ensures that all necessary registers are saved, it may result in unnecessary saves and restores, in case the particular registers are not in the active scope of the caller.

2.3 Register Usage Convention

Given that we want to reduce the number of registers being saved/restored during a function call, and also given that saving all registers by caller or all registers by callee is not appropriate (as discussed), an appropriate approach is to define a convention that specifies which registers are saved by the caller and the callee.

- **x5–x7, x28–x31: Temporary Registers**
  - Not preserved by the callee.
  - The callee is free to use them as per the need.
  - The caller saves them if needed.

- **x8–x9, x18–x27: Saved Registers**
  - If needed, the callee ensures to save these register values and restores them back before exiting the function.

The convention for all the registers is as follows:

Register	ABI Name	Description	Saver
x0	zero	Zero constant	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Fn args/return values	Caller
x12-x17	a2-a7	Fn args	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller

- **x3: Global Pointer (gp)**
  - Points to the start of the data segment.
  - Avoid overwriting this register.
- **x4: Thread Pointer (tp)**
  - Avoid overwriting this register.
- **x8: Frame Pointer (fp)**
  - Points to the beginning of the frame corresponding to the function called.
  - Helps in generating failure logs (e.g., error in line 48 of `f1()`, called by `f2()`, called by `main()`).
- **x10–x17: Argument Registers**
  - Used to pass arguments to the function.
  - If there are more than 8 arguments, the stack is used to pass additional arguments.
- **x10, x11: Return Value Registers**
  - Used to return the value from a function call.
  - In C programs, typically only one value is returned (arrays or structures are returned as pointers).
  - Two registers are allocated for the return value in case the value is very large.

Even though the hardware does not impose any restrictions in the usage of registers (any register can be used for any purpose), a convention is defined to support software compatibility and use of libraries. This convention is called ABI (application binary interface) and code/libraries following this convention are compatible to each other. As shown in the table, each register also has an alias name which corresponds to its usage as per the register conventions/ABI. For example, x10 can also be called a0.

2.4 Example

C Code

```
long long int leaf_example(long long int g, long long int h,
                           long long int i, long long int j){
    long long int f;
    f = (g + h) - (i + j);
    return f;
}
```

Arguments g to j are in x10 to x13, f is in x20, temporaries are in x5, x6.

RISC-V Assembly Comparison

Without Calling Convention	**Following the Calling Convention**
addi sp, sp, -24	addi sp, sp, *-8*
sd x5, 16(sp)	~~sd x5, 16(sp)~~
sd x6, 8(sp)	~~sd x6, 8(sp)~~
sd x20, 0(sp)	sd x20, 0(sp)
add x5, x10, x11	add x5, x10, x11
add x6, x12, x13	add x6, x12, x13
sub x20, x5, x6	sub x20, x5, x6
addi x10, x20, 0	addi x10, x20, 0
ld x20, 0(sp)	ld x20, 0(sp)
ld x6, 8(sp)	~~ld x6, 8(sp)~~
ld x5, 16(sp)	~~ld x5, 16(sp)~~
addi sp, sp, 24	addi sp, sp, *8*
jalr x0, 0(x1)	jalr x0, 0(x1)

In the above example, we don't need to store registers x5, x6 as they are temporary registers which will be stored by the caller if needed. However, x20 is a saved register which needs to be called by the callee if used.

3 Non-Leaf Procedures

3.1 Introduction

Leaf procedures (ones that do not call any other procedures) always behave as 'callee'. However, certain procedures may exist that are called by some procedure(s) $f_1(\cdot)$, and they themselves call some other procedure(s) $f_2(\cdot)$.

When such a non-leaf procedure is called by another function(procedure), say main(), the register x_1 gets updated with the return address of $f_1(\cdot)$. When $f_1(\cdot)$ subsequently calls $f_2(\cdot)$, the return address of $f_2(\cdot)$ must then be stored in x_1 (since this is our return register). However, this involves overwriting the previous address, which is essential for us to return. Thus, we must first store the value in x_1 before we call $f_2(\cdot)$.

3.2 Examples

3.2.1 Toy Example

C code

```
long long int f1 (long long int a, long long int b,
                  long long int c) {
    int x, y;
    x = a - f2(b);
    y = x + c;
    return y;
}
long long int f2 (long long int m){
    return m*8 + 5;
}
```

- Argument *a*, *b*, *c* in *x10*, *x11*, *x12*
- Result *y* in *x10*

```
// a: x10, b: x11, c: x12
f1: addi sp, sp, -48
    sd x1, 24(sp)
    sd x10, 16(sp)
    sd x11, 8(sp)
    sd x12, 0(sp)
    addi x10, x11, 0
    jal x1, f2
    addi x5, x10, 0
    ld x12, 0(sp)
    ld x11, 8(sp)
    ld x10, 16(sp)
    ld x1, 24(sp)
    addi sp, sp, 32
    sub x6, x10, x5
    add x10, x6, x12
    addi sp, sp, 16
    jalr x0, 0(x1)

f2: slli x6, x10, 3
    addi x10, x6, 5
    jalr x0, 0(x1)
```

Note:
x5: This register is used as a temporary register to hold the value of x10 after the function call to f2. Since it is not modified further or used after its initial assignment, there's no need to save it.
x6: This register is used in the function f2 to store intermediate results. Since it(with its current value) is not used outside the function, saving it is unnecessary. Both registers are used locally, and their values are not preserved across function calls.

x11, x12: It would help if the 'caller' f1 already had the information that 'callee' f2 doesn't use registers x11 and x12 at all. Thus, storing the values in these registers could've been avoided.

3.2.2 Factorial of a natural number - a recursive function

```
// if (n < 1) return (1);
// else return n * fact(n - 1);

fact:
  addi sp,sp,-16          // Save return address and n on stack
  sd x1,8(sp)
  sd x10,0(sp)
  addi x5,x10,-1          // x5 = n - 1
  bge x5,x0,L1            // if n-1 >= 0, go to L1
  addi x10,x0,1           // Else, set return value to 1
  addi sp,sp,16           // Pop stack, don't bother restoring values
  jalr x0,0(x1)           // Return

L1:
  jal x1,fact             // call fact(n-1)
  addi x6,x10,0           // move result of fact(n-1) to x6
  ld x10,0(sp)            // Restore caller's n
  ld x1,8(sp)             // Restore caller's return address
  addi sp,sp,16           // Pop stack
  mul x10,x10,x6          // n * fact(n-1)
  jalr x0,0(x1)           // return
```

Note: mul hasn't been discussed as a part of the basic instruction set. It is a part of the extension 'M' (integer multiply, divide, remainder). It has been used here to reduce the code length.

3.2.3 String copy

```
// Null-terminated string: C code
void strcpy(char x[], char y[])
{
    size_t i;
    i = 0;
    while ((x[i] = y[i]) != '\0')
        i += 1;
}


strcpy:
  addi sp,sp,-8           // adjust stack for 1 doubleword
  sd x19,0(sp)            // push x19
  add x19,x0,x0           // i=0
L1: add x5,x19,x10        // x5 = addr of y[i]
  lbu x6,0(x5)            // x6 = y[i]
```

```
    add x7,x19,x10          // x7 = addr of x[i]
    sb x6,0(x7)             // x[i] = y[i]
    beq x6,x0,L2            // if y[i] == 0 then exit
    addi x19,x19,1          // i = i + 1
    jal x0,L1               // next iteration of loop
L2: ld x19,0(sp)            // restore saved x19
    addi sp,sp,8            // pop 1 doubleword from stack
    jalr x0,0(x1)           // and return
```

4 Some Other RISC-V Instructions

1. The `auipc` instruction in RISC-V is used to compute addresses relative to the program counter (PC). The syntax is as follows:

   $$\texttt{auipc}\ rd, imm$$

   where:

   - `rd`: The destination register where the result will be stored.
   - *imm*: A 20-bit immediate value.

   The operation performed by the `auipc` instruction can be expressed as:

   $$\texttt{rd} \leftarrow \texttt{PC} + (imm \ll 12)$$

   Here, the immediate value *imm* is shifted left by 12 bits (equivalent to multiplying by 2^{12}), and the result is added to the current PC. Thus, this is used for generating addresses for jumps and branches relative to the current position of the PC.

2. slt, sltu, slti, sltui
   Used to set `rd` to 1 if the comparison (signed/unsigned) between `rs1` and `rs2`/*imm* is true, else set it to 0.

3. addw, subw, addiw
   Used to perform 32-bit addition/subtraction (with immediate) and sign-extend the result to 64 bits.

4. sllw, srlw, slliw, srliw, sraiw
   Used to perform 32-bit logical/arithmetical left/right shifts and sign-extend the result to 64 bits.

The 'w' effectively signifies a word (4 bytes or equivalently, 32 bits) in the above instructions.